Assignment 1

Pollux Taal - 8254087

Samuel Stork - 4410521

October 2026

1 Binary Line detection

For this analysis a pipeline has been implemented consisting of multiple operations on the input image. First we apply gaussian smoothing, this is done using a kernel size of 5 and a sigma-value of 1. After this we apply edge detection and apply a threshold ($t_{mag} \in \{100, 160, 220\}$). Finally we perform binary Hough transform and visualize the most salient line segments (using a threshold $t_{peaks} \in \{160, 220\}$) using red on top of the binary edge map.

In our app you can adjust t_{mag} using Threshold (0), t_{peak} using Threshold (1), and minimum line intensity using Threshold (2).

The Hough line segmentation was done using a minimum segment length of 10, maximum gap of 30 and a minimum intensity of 0 (effectively 1 since background pixels get ignored).

Images were generated using combinations of different thresholding values, totaling 6 images. The parameters t_{mag} and t_{peak} are defined over the following sets:

$$t_{\text{mag}} \in \{100, 160, 220\}, \qquad t_{\text{peak}} \in \{160, 220\}.$$

Images are generated using a pair of these thresholding values (t_{mag},t_{peak}).

As shown in Figure 1e, the parameter combination t_{mag} value of 160 with a t_{peak} value of 160 seems to produce the greatest number of long, accurate, straight lines. Assuming that maximizing the number of such lines is the primary objective, this combination is selected as the optimal set of parameters among those evaluated.

2 Grayscale Line Detection

The grayscale pipeline had more alterations than simply changing the Hough transform to grayscale. As the accumulator array values can exceed 255, we adjust the contrast to match the byte range. We do this after potentially normalising the accumulator. All images were generated with a minimum line

segment length of 30 and a maximum gap length of 10. The modified variables were the presence of normalisation, gaussian kernel size and corresponding sigma, peak threshold, and minimum intensity during segment computation.

The outputs for the parameter combinations can be seen in figure 2. We believe that figure 2e, the image showing a kernel size of 7, a sigma of 1.5, a t_{peak} of 175 and a minimum intensity of 100 presents the best balance between avoiding spurious line segments while capturing the most important ones.

We find that the thresholding process of the binary pipeline is beneficial in reducing noise which can cause spurious captured lines. We find that the minimum intensity acts similarly to the overall intensity threshold. Furthermore, the grayscale pipeline is considerably more computationally intensive, though a large part of that is the normalisation.

3 Choice Tasks

In this section we discuss the choice tasks we have completed.

3.1 Flood Fill Algorithm

One of the choice tasks we implemented was the flood fill algorithm, this algorithm fills regions with a uniform color so that clear distinctions can be made between regions. This algorithm can be found in the drop-down menu under the name "FloodFill".

For this assignment the algorithm has been implemented using binary images as input, the output consists of a grayscale image with different regions consisting of distinctive shades of gray, made more clear by applying contrast adjustment (see figure 3).

The algorithm has been made using a recursive method, where foreground pixels are colored, after which we recursively visit their neighbors.

3.2 Normalise the Counts in the Accumulator Array

The effect of normalisation can be observed in our task 2, and direct comparisons can be seen in figure 2. Without normalisation, the lines detected are highly biased towards the larger image dimension. This is also visible when comparing accumulator arrays as seen in figure 4.

We achieved normalisation by dividing the accumulator array by the accumulator array corresponding to a foreground-only image of the same dimensions as the input.

3.3 Circle Transform

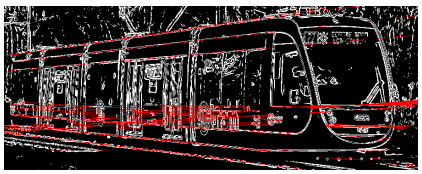
We partially implemented the Hough transform for circles, but did not complete the visualisation.



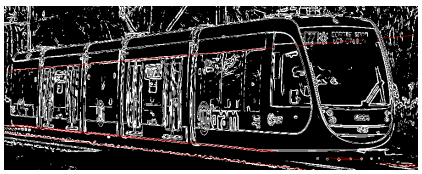
(a) Image A before applying grayscale and contrast adjustment



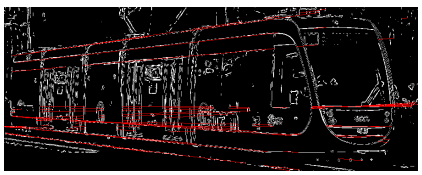
(b) Image A after applying grayscale and contrast adjustment



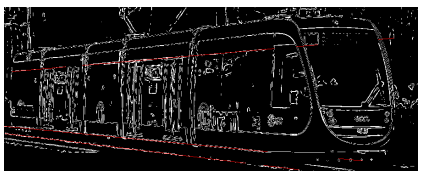
(c) Image A after pipeline with $t_{mag} = 100$ and $t_{peak} = 160$



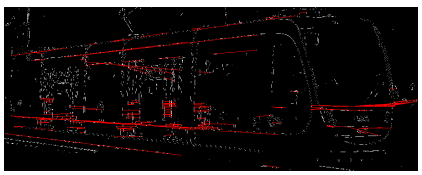
(d) Image A after pipeline with $t_{mag} = 100$ and $t_{peak} = 220$



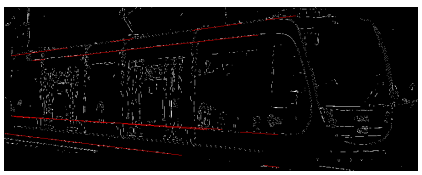
(e) Image A after pipeline with $t_{mag} = 160$ and $t_{peak} = 160$



(f) Image A after pipeline with $t_{mag} = 160$ and $t_{peak} = 220$

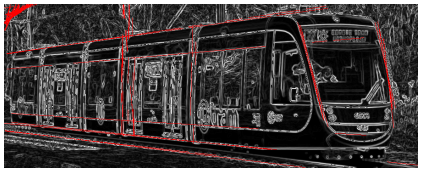


(g) Image A after pipeline with $t_{mag} = 220$ and $t_{peak} = 160$

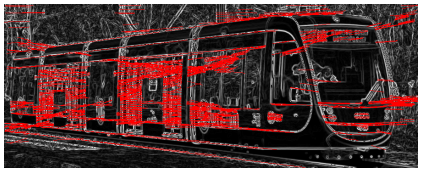


(h) Image A after pipeline with $t_{mag} = 220$ and $t_{peak} = 160$

Figure 1: The images for binary line detection



(a) Grayscale pipeline with kernel size 5, sigma 1.0, peak threshold 175, minimum intensity 100, normalised



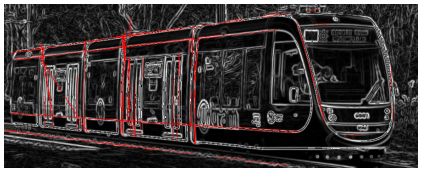
(b) Grayscale pipeline with kernel size 5, sigma 1.0, peak threshold 175, minimum intensity 100, not normalised



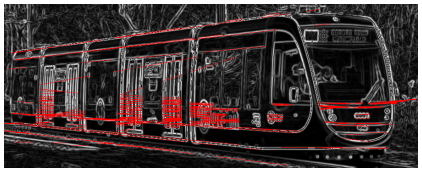
(c) Grayscale pipeline with kernel size 7, sigma 1.5, peak threshold 120, minimum intensity 100, normalised



(d) Grayscale pipeline with kernel size 7, sigma 1.5, peak threshold 120, minimum intensity 100, not normalised



(e) Grayscale pipeline with kernel size 7, sigma 1.5, peak threshold 175 minimum intensity 100, normalised



(f) Grayscale pipeline with kernel size 7, sigma 1.5, peak threshold 175, minimum intensity 100, not normalised

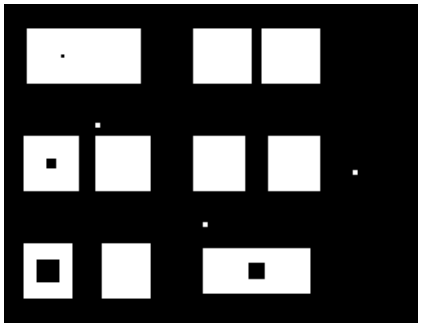


(g) Grayscale pipeline with kernel size 7, sigma 1.5, peak threshold 175, minimum intensity 150, normalised

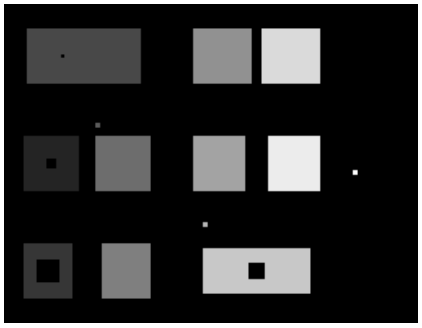


(h) Grayscale pipeline with kernel size 7, sigma 1.5, peak threshold 175, minimum intensity 150, not normalised

Figure 2: The images for grayscale line detection, with and without accumulator normalisation

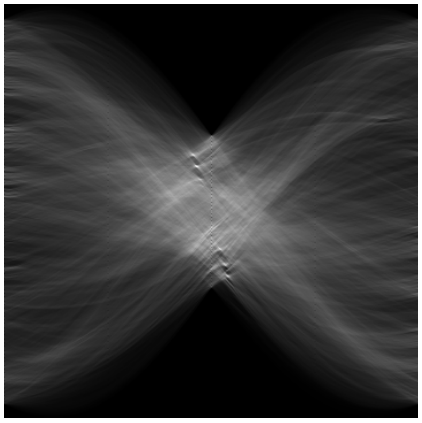


(a) Original image; before applying flood-fill

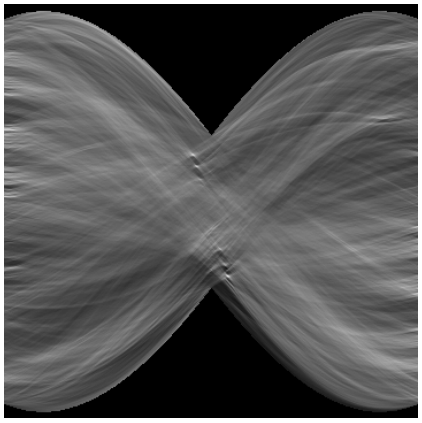


(b) Image after the application of flood-fill

Figure 3: Images before and after the application of the flood-fill algorithm



(a) Original accumulator array



(b) Normalised accumulator array

Figure 4: Accumulator array before and after the application of the normalisation algorithm